TASK 17 — FILTERING AND SORTING DATAFRAMES

_

Description: Filter rows using conditions and sort DataFrame values in ascending and descending order.

Objective: Learn fundamental techniques for retrieving specific records from datasets.

1. Tools

• Python
• Pandas
• Jupyter Notebook

2. Deliverables

• At least five filtering examples
• Multiple sorting examples
• A short analysis of filtered records

3. Dataset Used

The project uses a student-performance dataset. It contains Student_ID, Name, Department, Python_Marks, SQL_Marks, Attendance, Projects, Status, and Average. Average is calculated from Python and SQL marks.

4. What Is a DataFrame?

A Pandas DataFrame is a two-dimensional table made of rows and columns. Each column represents a variable and each row represents a record. Filtering and sorting are two fundamental operations used to explore this table.

5. What Is Filtering?

Filtering means selecting only the rows that satisfy a condition. Pandas evaluates the condition for every row and returns the rows where the condition is True. Filtering is useful when a large dataset contains many records but the analysis needs only a specific subset.

Basic filtering syntax

```
df[df["column"] >= value]
```

6. Step-by-Step: Single-Condition Filtering

Step 1: Choose the column. Step 2: Define the condition. Step 3: Put the condition inside df[ ]. Example: select students with Python marks of 80 or above.

```
high_python = df[df["Python_Marks"] >= 80]
```

Meaning: Pandas checks every value in Python_Marks and keeps only rows where the value is at least 80.

7. Filtering Using Attendance

The same method can be applied to attendance. This can identify students meeting an attendance requirement.

```
df[df["Attendance"] >= 85]
```

8. Filtering Text / Categorical Values

Text columns can also be filtered. For example, Department can be compared with the text value CSE.

```
df[df["Department"] == "CSE"]
```

9. Multiple Conditions — AND

Use the & operator when all conditions must be True. Each condition should be enclosed in parentheses. Example: students must have Average >= 80 AND Attendance >= 85.

```
df[(df["Average"] >= 80) & (df["Attendance"] >= 85)]
```

10. Multiple Conditions — OR

Use the | operator when either condition can be True. Example: select students who scored at least 90 in Python OR SQL.

```
df[(df["Python_Marks"] >= 90) | (df["SQL_Marks"] >= 90)]
```

11. NOT Condition

The ~ operator reverses a Boolean condition. It can be used when the analysis needs records that do not match a condition.

```
df[~(df["Status"] == "Inactive")]
```

12. Combining Text and Numerical Conditions

Filtering becomes more powerful when categorical and numerical conditions are combined. For example, find CSE students whose SQL marks are at least 80.

```
df[(df["Department"] == "CSE") & (df["SQL_Marks"] >= 80)]
```

13. What Is Sorting?

Sorting arranges rows according to the values in one or more columns. It is useful for ranking students, finding the highest or lowest values, and making results easier to interpret.

14. Ascending Sorting

Ascending order goes from the smallest value to the largest value. This is the default behavior of sort_values().

```
df.sort_values(by="Average")
```

15. Descending Sorting

Descending order goes from the largest value to the smallest value. Use ascending=False when the highest values should appear first.

```
df.sort_values(by="Average", ascending=False)
```

16. Sorting by Attendance

The same technique can rank students by attendance.

```
df.sort_values(by="Attendance", ascending=False)
```

17. Sorting by Multiple Columns

More than one column can be supplied using a list. Here Department is sorted in ascending order, and Average is sorted from highest to lowest inside each department.

```
df.sort_values(by=["Department", "Average"], ascending=[True, False])
```

18. Filtering First, Then Sorting

Filtering and sorting can be combined in one workflow. First select students with attendance >= 85, then rank those students by Average from highest to lowest.

```
df[df["Attendance"] >= 85].sort_values(by="Average", ascending=False)
```

19. Top and Bottom Records

After sorting, head() can be used to retrieve the first few records. This is useful for finding top performers or the lowest-performing records.

```
top_3 = df.sort_values(by="Average", ascending=False).head(3)
bottom_3 = df.sort_values(by="Average").head(3)
```

20. Required Filtering Examples

No.	Filtering task	Example
1	Python marks ≥ 80	df[df["Python_Marks"] >= 80]
2	Attendance ≥ 85	df[df["Attendance"] >= 85]
3	Average ≥ 80 AND attendance ≥ 85	(df["Average"] >= 80) & (df["Attendance"] >= 85)
4	Python ≥ 90 OR SQL ≥ 90	(df["Python_Marks"] >= 90) | (df["SQL_Marks"] >= 90)
5	CSE AND SQL ≥ 80	(df["Department"] == "CSE") & (df["SQL_Marks"] >= 80)
6	Active students	df[df["Status"] == "Active"]

21. Required Sorting Examples

No.	Sorting task	Example
1	Average ascending	df.sort_values(by="Average")
2	Average descending	df.sort_values(by="Average", ascending=False)
3	Attendance descending	df.sort_values(by="Attendance", ascending=False)
4	Department ↑, Average ↓	df.sort_values(by=["Department","Average"], ascending=[True,False])
5	Filter attendance, then sort average	df[df["Attendance"] >= 85].sort_values(by="Average", ascending=False)

22. Boolean Operators — Quick Explanation

& means AND: every condition must be True.
| means OR: at least one condition must be True.
~ means NOT: reverses the condition.
For Pandas Series, use these operators instead of Python's and/or for element-by-element conditions.

23. Common Mistakes and How to Avoid Them

1. Do not write df[df['A'] > 50 and df['B'] > 50]. Use & with parentheses.
2. Do not forget parentheses around each condition.
3. Use the exact DataFrame column name.
4. Remember that ascending=False is required for descending order.
5. Do not confuse filtering with sorting: filtering selects rows, while sorting changes their order.
6. When using multiple sort columns, provide the matching ascending list if different directions are required.

24. Efficient Data Analysis Workflow

Step 1 — Load the dataset.
Step 2 — Inspect columns and values.
Step 3 — Define the question you want to answer.
Step 4 — Build the Boolean condition.
Step 5 — Filter the required records.
Step 6 — Sort the result when ranking is needed.
Step 7 — Select only useful columns for reporting.
Step 8 — Interpret the result.

Example workflow

```
result = (df[df["Attendance"] >= 85]
          .sort_values("Average", ascending=False)
          [["Student_ID", "Name", "Average", "Attendance"]])
```

25. Practical Student Analysis

The dataset contains **10** students. The overall mean of Average is **79.55**. The highest Average is **94.00** and the lowest is **63.00**. **6** students have attendance of at least 85%.

26. Interpretation of Filtered Records

Filtering does not change the original question into a ranking problem. It first identifies the records that satisfy the rule. Sorting can then organize those selected records. For example, attendance >= 85 identifies eligible students, while sorting Average descending shows the strongest performers among those eligible students.

27. Real-World Applications

• Education — filter students by marks, attendance, department, or status.
• Sales — filter high-value orders and sort them by revenue.
• HR — filter employees by department or performance and rank them.
• E-commerce — identify low-stock products and sort by sales.
• Customer analytics — identify customers meeting spending or activity conditions.

28. Complete Practical Program

```
import pandas as pd

# Load the dataset
df = pd.read_csv("student_performance.csv")

print("=== ORIGINAL DATA ===")
print(df)


# PART A: FILTERING


# 1. Python marks >= 80
filter_1 = df[df["Python_Marks"] >= 80]
print("\n1. Python marks >= 80")
print(filter_1[["Student_ID", "Name", "Python_Marks"]])

# 2. Attendance >= 85
filter_2 = df[df["Attendance"] >= 85]
print("\n2. Attendance >= 85")
print(filter_2[["Student_ID", "Name", "Attendance"]])

# 3. Average >= 80 AND attendance >= 85
filter_3 = df[
    (df["Average"] >= 80) &
    (df["Attendance"] >= 85)
```

```
]
print("\n3. Average >= 80 AND Attendance >= 85")
print(filter_3[["Student_ID", "Name", "Average", "Attendance"]])

# 4. Python >= 90 OR SQL >= 90
filter_4 = df[
    (df["Python_Marks"] >= 90) |
    (df["SQL_Marks"] >= 90)
]
print("\n4. Python >= 90 OR SQL >= 90")
print(filter_4[["Student_ID", "Name", "Python_Marks", "SQL_Marks"]])

# 5. CSE students with SQL >= 80
filter_5 = df[
    (df["Department"] == "CSE") &
    (df["SQL_Marks"] >= 80)
]
print("\n5. CSE AND SQL >= 80")
print(filter_5[["Student_ID", "Name", "Department", "SQL_Marks"]])

# 6. Active students
filter_6 = df[df["Status"] == "Active"]
print("\n6. Active students")
print(filter_6[["Student_ID", "Name", "Status"]])


# PART B: SORTING


# 7. Average ascending
sort_1 = df.sort_values(by="Average")
print("\n7. Average - Ascending")
print(sort_1[["Student_ID", "Name", "Average"]])

# 8. Average descending
sort_2 = df.sort_values(by="Average", ascending=False)
print("\n8. Average - Descending")
print(sort_2[["Student_ID", "Name", "Average"]])

# 9. Attendance descending
sort_3 = df.sort_values(by="Attendance", ascending=False)
print("\n9. Attendance - Descending")
print(sort_3[["Student_ID", "Name", "Attendance"]])

# 10. Multiple-column sorting
sort_4 = df.sort_values(
    by=["Department", "Average"],
    ascending=[True, False]
)
print("\n10. Department ascending, Average descending")
print(sort_4[["Student_ID", "Name", "Department", "Average"]])


# PART C: FILTER + SORT


result = (
    df[df["Attendance"] >= 85]
    .sort_values(by="Average", ascending=False)
)

print("\n11. Attendance >= 85, sorted by Average descending")
print(result[["Student_ID", "Name", "Average", "Attendance"]])

# Top 3
top_3 = df.sort_values(by="Average", ascending=False).head(3)
print("\n12. Top 3 students")
print(top_3[["Student_ID", "Name", "Average"]])

# Bottom 3
bottom_3 = df.sort_values(by="Average").head(3)
print("\n13. Bottom 3 students")
print(bottom_3[["Student_ID", "Name", "Average"]])


# PART D: SHORT ANALYSIS


print("\n=== SHORT ANALYSIS ===")
print("Number of students:", len(df))
print("Overall average:", round(df["Average"].mean(), 2))
print("Highest average:", df["Average"].max())
print("Lowest average:", df["Average"].min())
print("Students with attendance >= 85:",
      (df["Attendance"] >= 85).sum())
```

29. Interview Questions and Answers

Q. How do you filter rows in Pandas?
A. Use Boolean indexing, for example df[df['Age'] >= 18].

Q. How do you sort a DataFrame?
A. Use sort_values(), for example df.sort_values(by='Salary').

Q. How do you sort in descending order?
A. Use ascending=False.

Q. How do you combine multiple conditions?
A. Use & for AND and | for OR, with parentheses around each condition.

Q. What is the difference between filtering and sorting?
A. Filtering selects rows that meet a condition. Sorting changes the order of rows.

Q. How do you sort by multiple columns?
A. Pass a list to by and, when needed, a list to ascending.

Q. Can filtering and sorting be chained?
A. Yes. A filtered DataFrame can immediately be passed to sort_values().

Q. Why is filtering important in data analysis?
A. It reduces a dataset to the records relevant to a particular question or business rule.

30. Quick Revision Notes

Filter: df[condition]
AND: (condition1) & (condition2)
OR: (condition1) | (condition2)
NOT: ~(condition)
Ascending: df.sort_values('column')
Descending: df.sort_values('column', ascending=False)
Multiple columns: df.sort_values(by=['A','B'], ascending=[True,False])
Top records: sorted_df.head(n)

31. Top 5 Skills Used

1. Pandas
2. DataFrame Filtering
3. Data Sorting & Ranking
4. Boolean Conditions
5. Python Data Analysis

32. Feedback

The task was very useful for understanding how Pandas can be used to retrieve relevant records and organize data. I learned how to apply single and multiple Boolean conditions, use AND and OR logic, sort values in ascending and descending order, and combine filtering with sorting. The practical student-performance dataset improved my confidence in performing efficient DataFrame analysis.